week4 예습과제

메모

5장. GBM (Gradient Boosting Machine)

- **핵심 아이디어**: 약한 학습기(보통 얕은 결정트리)를 **순차적으로** 학습시키며, 이전 모델이 틀린 부분(잔차/손실의 기울기)을 다음 모델이 보완하도록 합산.
- **편향-분산 관점**: 배깅(랜덤포레스트)은 분산 감소, 부스팅(GBM)은 **편향 감소**에 강점(대신 과적합/튜닝 민감).
- **손실 함수 기반 학습**: 회귀/분류 등 문제에 맞는 손실을 최소화하도록 단계적으로 모델을 추가.
- **대표 하이퍼파라미터(개념)**
  - `n_estimators`: 트리 개수(성능↑ 가능, 과적합/시간↑)
  - `learning_rate`: 학습률(작을수록 안정적, 보통 `n_estimators`↑)
  - `max_depth` / `max_leaf_nodes`: 트리 복잡도(과적합 핵심 레버)
  - `subsample`: 행 샘플링으로 과적합 완화
  - `min_samples_split`, `min_samples_leaf`: 리프 최소 샘플로 과적합 완화

6장. XGBoost (eXtra Gradient Boost)

- **장점**: GBM을 실전적으로 개선(정규화, 결측치 처리, 속도 최적화, 조기 종료 등).
- **주요 개념**
  - 목적함수 = 손실(loss) + 정규화(트리 복잡도 페널티)
  - 2차 테일러 근사(gradient+hessian) 활용
- **파이썬 래퍼 vs 사이킷런 래퍼**
  - 파이썬 래퍼: `DMatrix`, `train`, `evals`, early stopping 등 XGBoost 고유 기능 활용 폭넓음
  - 사이킷런 래퍼: `XGBClassifier/Regressor`로 파이프라인·CV 결합 쉬움
- **대표 하이퍼파라미터(핵심만)**
  - 학습: `n_estimators`, `learning_rate`, `subsample`, `colsample_bytree`

- 복잡도: `max_depth`, `min_child_weight`, `gamma`
- 정규화: `reg_alpha` (L1), `reg_lambda` (L2)
- 불균형: `scale_pos_weight`

7장. LightGBM

- **특징**: 대용량/고차원에서 빠르게 동작하도록 설계된 GBDT 계열(히스토그램 기반 분할 등).
- **리프 중심(leaf-wise) 성장**을 자주 사용: 성능↑ 가능하지만 과적합 주의.
- **대표 하이퍼파라미터(핵심만)**
  - 복잡도: `num_leaves`, `max_depth`, `min_data_in_leaf`
  - 학습: `n_estimators`, `learning_rate`, `subsample`, `colsample_bytree`
  - 정규화: `lambda_l1`, `lambda_l2`
  - 불균형: `is_unbalance` 또는 `scale_pos_weight`

8장. 베이지안 최적화 기반 튜닝 (HyperOpt)

- **그리드/랜덤서치 한계**: 조합 폭발/비효율 또는 시행착오 비용 큼.
- **베이지안 최적화**: 과거 실험 결과로 다음 탐색 지점을 똑똑하게 선택 → **적은 실험으로 좋은 조합** 탐색.
- **HyperOpt 기본 흐름**

  1) 탐색공간 정의 → 2) 목적함수 정의(CV 점수 등) → 3) TPE 등으로 반복 탐색 → 4) 최적 파라미터 재학습/검증
- **팁**: 검증전략(CV/홀드아웃) 고정, 누수 방지, early stopping 병행.

10장. 신용카드 사기 검출 (불균형 데이터)

- **특성**: 양성(사기) 비율이 매우 낮아 정확도만으로는 평가가 왜곡.
- **지표**: Precision/Recall/F1/PR-AUC 등을 함께 고려(업무 목표에 따라 우선순위 결정).
- **대표 접근**
  - 언더샘플링: 빠르지만 정보 손실
  - 오버샘플링: 과적합 위험(단순 복제)
  - SMOTE: 합성 샘플 생성(경계/노이즈 민감 가능)

    - 분포 변환/이상치 처리: 학습 안정성 개선
- **중요**: 샘플링은 학습 데이터에만 적용, threshold 조정으로 P-R trade-off 맞추기.

11장. 스태킹 앙상블

- **핵심 아이디어**: 여러 base 모델 예측값을 입력으로 메타모델이 최종 예측.
- **CV 기반 스태킹이 중요한 이유**: 누수/과적합 방지를 위해 OOF(out-of-fold) 예측으로 메타모델 학습.
- **팁**: base 모델 다양성 확보, 메타모델은 단순하게 시작, CV 설계 엄격히.

분류·결정트리·앙상블·랜덤포레스트